

# RSA

COME FUNZIONA?

I SIGNORE DELLA TRUFFA

$$\begin{aligned} a &\equiv b \pmod{\varphi(n)} \\ n^a &\equiv n^b \pmod{n} \end{aligned}$$

COME SI GENERA UNA CHIAVE? FUNDAMENTALE!

$p, q$  PRIMI CASUALI DI  $k/2$  BIT

costruire  $n \leftarrow pq$

scegli  $e$  a caso t.c.  $\text{gcd}(e, \varphi(n)) = 1$   $e \in \mathbb{Z}_{\varphi(n)}^*$

definire  $d = e^{-1} \pmod{\varphi(n)}$   $d \cdot e \equiv 1 \pmod{\varphi(n)}$

PUB  $P_K$   $n, e$

PRIV  $S_K$   $n, d$

$\varphi(n) = (p-1)(q-1)$  è il numero di euler

$d$  esiste perché  $e$  essendo coprimo con  $\varphi(n)$ , è un elemento  $\mathbb{Z}_{\varphi(n)}^*$   
ammette inversa in  $\varphi(n)$

$$\begin{aligned} (m^e)^d &= m^{e \cdot d} \\ &= m^{1 + k \varphi(n)} \\ &= m * (m^{\varphi(n)})^k \quad // \text{TEO DI EULERO} \\ &\quad \text{(numero elevato alla cardinalità di un gruppo } \Rightarrow 1) \\ &= m \quad \text{CORRETTI!} \end{aligned}$$

però non dimostra che RSA funziona, questo funziona quando  $m \in \mathbb{Z}_n^*$   $\Rightarrow$  TEO RESIDUO CINESE  
 $d$  è l'informazione TRAPPOLA

CHI NON HA LA CHIAVE PRIVATA, SA DECODIFICARE?  
no

DM \*

$$a \equiv b \pmod{\varphi(n)}$$

$\Leftrightarrow$

$$a = b + l \varphi(n) \quad \exists l$$

$$m^{b+l\varphi(n)} = m^a$$

||

$$m^b * (m^{\varphi(n)})^l = m^a$$

RSA È SICURO? SI RIESCONO A DECODIFICARE I MESSAGGI?

MESSAGGIO PICCOLO

$$m^2 < n \quad // \text{facilmente decodificabili}$$

MESSAGGI SPARSI

INFO PARZIALI

È POLINOMIALE SIO ALGO?

algo in  $k^4$  probabilistico e polinomiale

$G: \{1^k\} \rightarrow S_K \quad P_K$  parametro di sicurezza (numero che garantisce più sicurezza, quanto è alto il valore di  $k$ , o meglio è più difficile rompere il sistema)

per avere algo poli nella rappresentazione del valore, l'input non può essere rappresentato in binario

ALGO DI CODIFICA E DECODIFICA

$$\begin{aligned} E: m, P_K &\longrightarrow c \\ &\longmapsto m^e \pmod{n} \quad (\text{nel gruppo } \mathbb{Z}_n^*) \end{aligned}$$

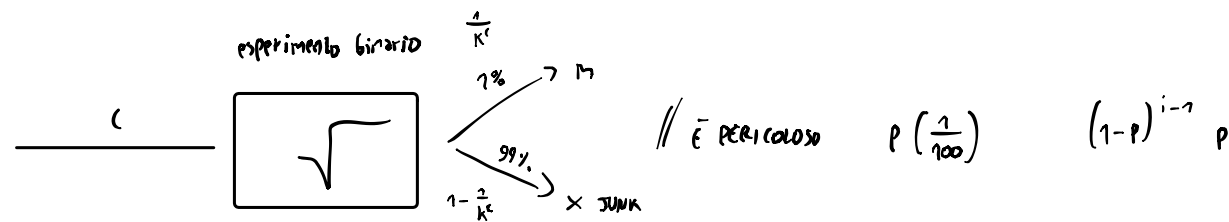
$$\begin{aligned} D: c, S_K &\longrightarrow m \\ &\longmapsto c^d \pmod{n} \end{aligned}$$

$$\forall m \quad D(E(m, P_K), S_K) = m$$

• RSA è sicuro perché non conosciamo un algoritmo probabilistico polinomiale per calcolare la radice e-esima di un numero.

↳ SE INVECE QUESTO ALGORITMO ESISTESSE CON PROBABILITÀ 1 SU 100?

Sarebbe un problema, perché in media se provo 100 volte l'algoritmo riesco ad ottenere il messaggio,



- fissando  $a \in \mathbb{Z}_n^*$
  - prendiamo  $R \in \mathbb{Z}_n^*$
  - calcoliamo  $a \cdot \underbrace{R^e}_{\text{è un elemento casuale di } \mathbb{Z}_n^*}$  è invertibile perché è biettiva
- $\left. \begin{array}{l} a \cdot R^e \text{ è un elemento distribuito in } \mathbb{Z}_n^*, \text{ perché } a \text{ è scelto uniformemente in } \mathbb{Z}_n^* \\ \text{poiché la moltiplicazione per } a \text{ è una funzione biettiva, perché } a \text{ ammette inverso} \end{array} \right\}$

• l'algoritmo è indipendente dall' $a$  fissato preso, e  $\frac{1}{100}$  riesco ad ottenere la radice e-esima

$$a \cdot R^e \longrightarrow \sqrt[e]{a \cdot R^e} = R^e \sqrt[e]{a} \cdot \frac{m \cdot R}{R} = m$$

- devo anche essere in grado di riconoscere dall'output dell'algoritmo se la risposta è corretta, per farlo basta rilevarlo alla  $e$
- un sistema diventa attaccabile se il tempo per attaccarlo è polinomiale, perché ripetendo l'algo  $K$  volte (aumentando  $\frac{1}{K}$  probabilità di successo) abbiamo una probabilità di successo 1

$$\underbrace{\forall A \in \text{PPR}}_{\text{attaccante}} \quad \forall c > 0 \quad \exists \bar{n} \quad \forall n > \bar{n} \quad \left. \begin{array}{l} \Pr [A \text{ calcola } \sqrt[e]{\cdot}] < n^{-c} \\ \text{non c'è quantificazione di } c \end{array} \right\} \begin{array}{l} \text{basta scegliere la chiave sufficientemente lunga per rendere la probabilità} \\ \text{di attacco più piccola di } n^{-c} \end{array}$$

•  $c$  rappresenta il livello di difficoltà

↳ Dato  $n$  è vero che non esiste un algoritmo per in grado di calcolare  $\sqrt[n]{a}$ ?

• se  $n$  fissato sì, perché c'è la fattorizzazione di  $n$   $\rightarrow$  informazione trapadata // informazione binaria da proteggere

• alla fine si pensa che non esiste un algoritmo per che dati  $n, e, a$  calcola  $\sqrt[n]{a}$  in  $\mathbb{Z}_p^*$